Digital Logic Design

Course code: CSE-2323
Credit Hour: 3

Md. Mujibur Rahman Maruf

Adjunct Lecturer,

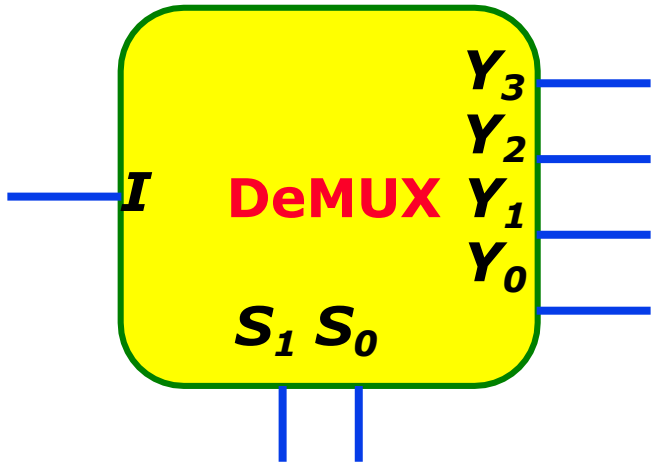
Dept. of CSE, IIUC

Mujiburmaruf.cuet17@gmail.com

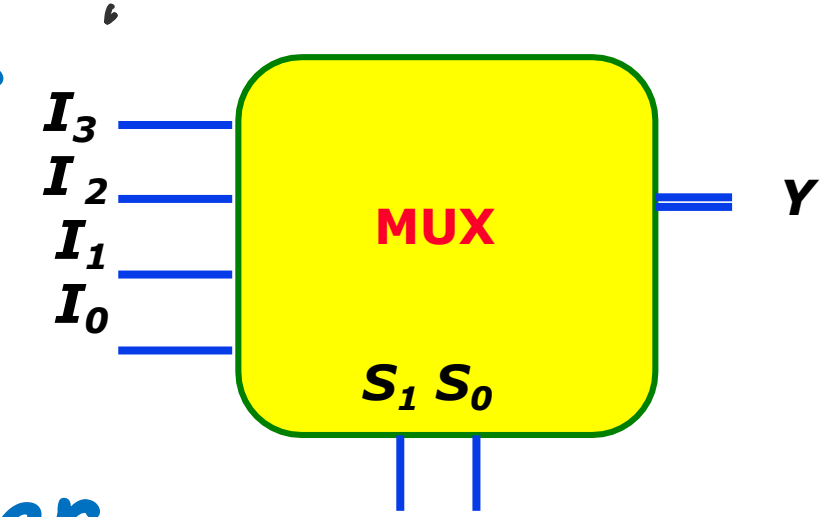
International Islamic University Chittagong(IIUC)

CSE_2323

Multiplexer & Demultiplexer



Multiplexer And Demuultiplexer



Mux (Multiplexer)

Multiplexer (MUX)

Routes one of many inputs to a single output
Also called a *selector*

A Digital Multiplexer (MUX) is a combinational Circuit that select one digital information from several sources and transmits the selected information on a single output line

Demultiplexer (DEMUX)

Routes a single input to one of many outputs
Also called a *decoder*

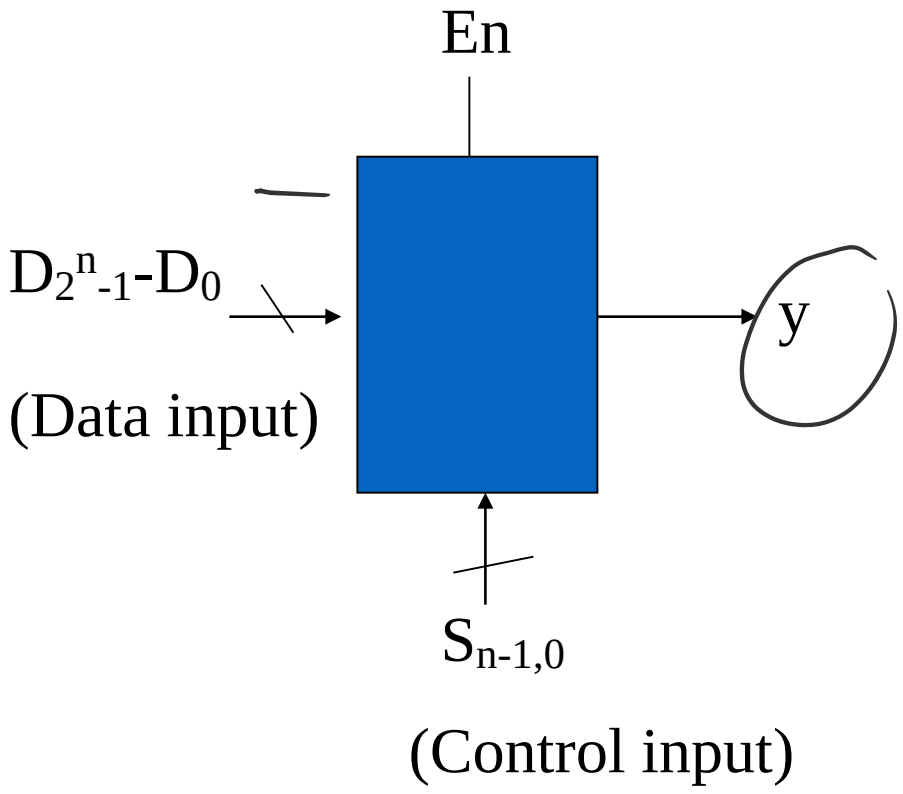
➢ Multiplexer contains the followings:

- ❖ data inputs
- ❖ selection inputs
- ❖ a *single output*

➢ Selection input determines the input that should be connected to output.

➢ The multiplexer acts like an electronic switch that selects one fro different.

Mux (Multiplexer)



Description

If En = 1

$y = D_i$ where $i = (S_{n-1}, .. , S_0)$

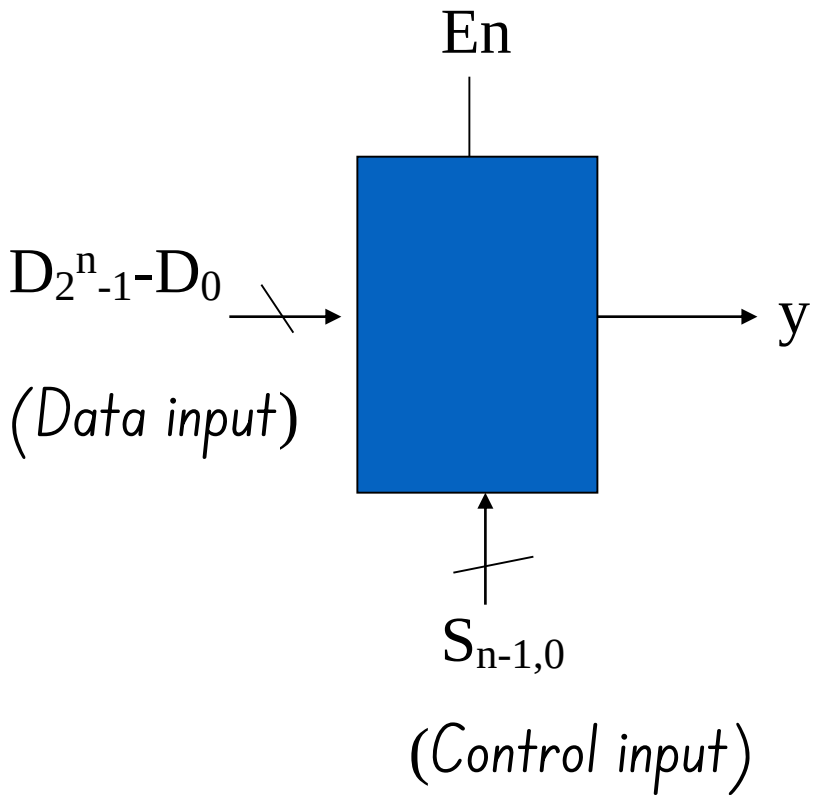
Else

$y = 0$

Basic concept

- 2^n data inputs; n control inputs ("selects"); 1 output
- Connects one of 2^n inputs to the output
- "Selects" decide which input connects to output
- Two alternative truth-tables: Functional and Logical

Mux (Multiplexer)



Description

If En = 1

$y = D_i$ where $i = (S_{n-1}, .. , S_0)$

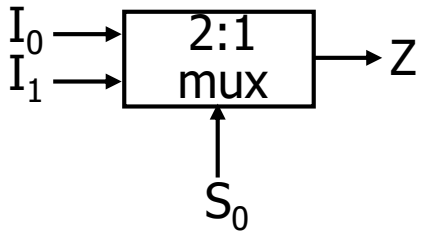
Else

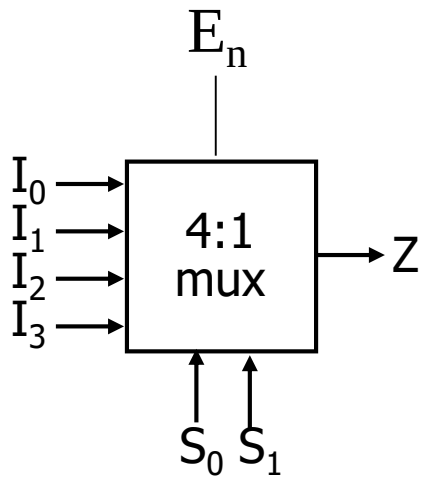
$y = 0$

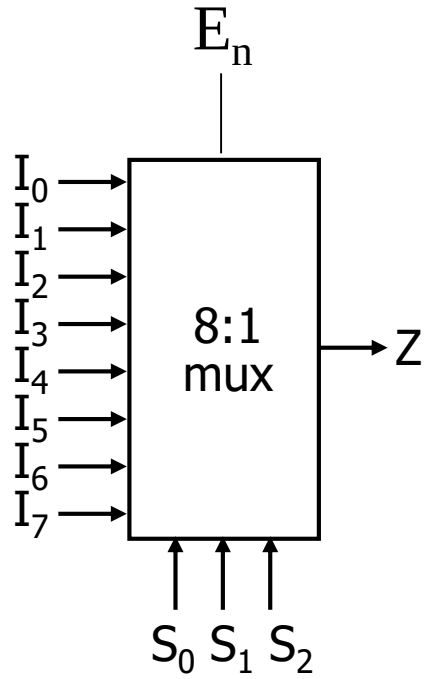
Basic concept

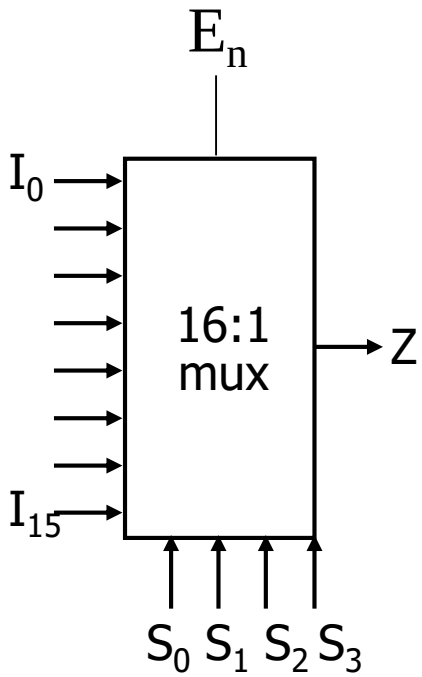
- 2^n data inputs; n control inputs ("selects"); 1 output
- Connects one of 2^n inputs to the output
- "Selects" decide which input connects to output
- Two alternative truth-tables: Functional and Logical

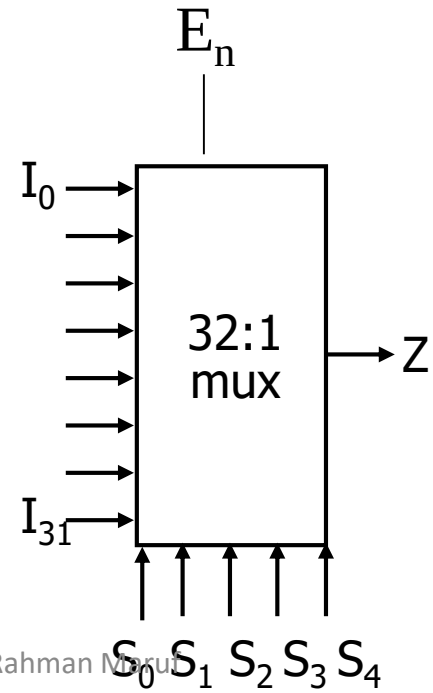
Multiplexers



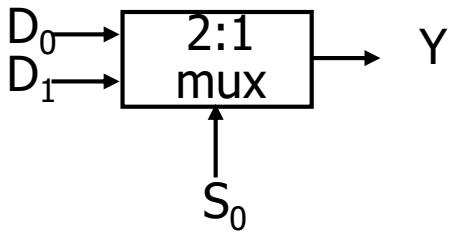


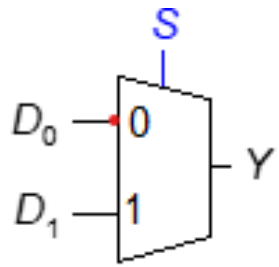






2x1 Multiplexer





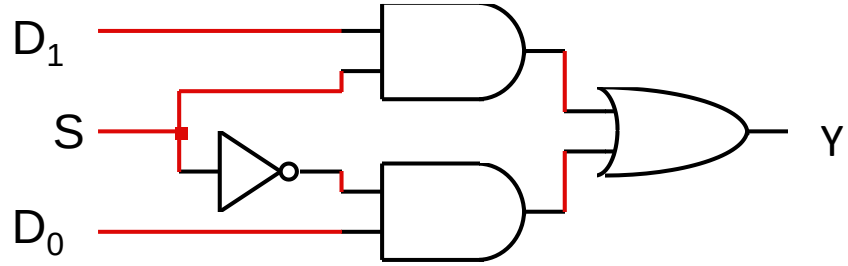
Functional truth table

S	Y
0	D_0
1	D_1

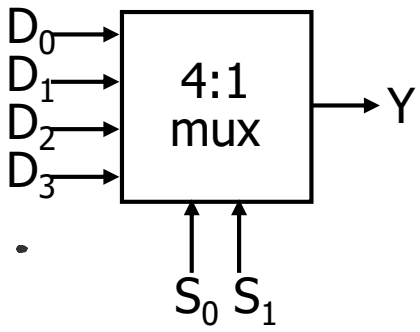
The 2×1 mux has two input lines, one output line, and a single selection line. The output of the 2×1 Mux will depend on the selection line S0,

- When S is 0(low), the I0 is selected
- when S0 is 1(High), I1 is selected

$$Y = SD_1 + S'D_0$$



4x1 Multiplexer

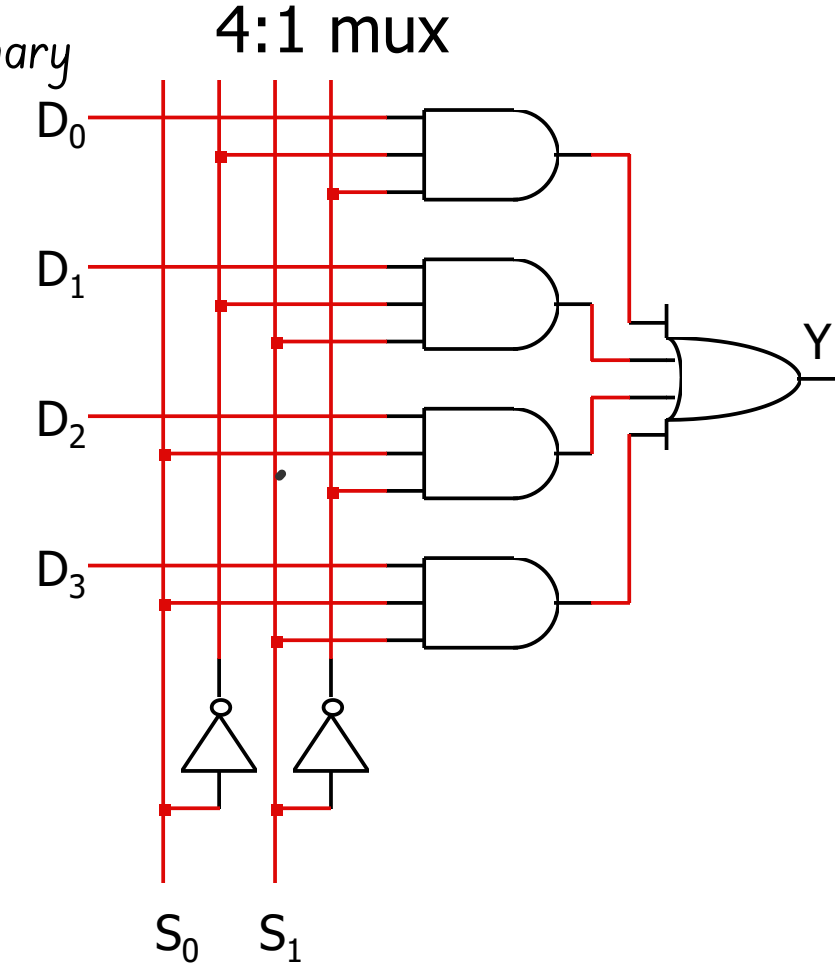


The output of the multiplexer is determined by the binary value of the selection lines

- When SISO=00, the input D0 is selected.
- When SISO=01, the input D1 is selected.
- When SISO=10, the input D2 is selected.
- When SISO=11, the input D3 is selected

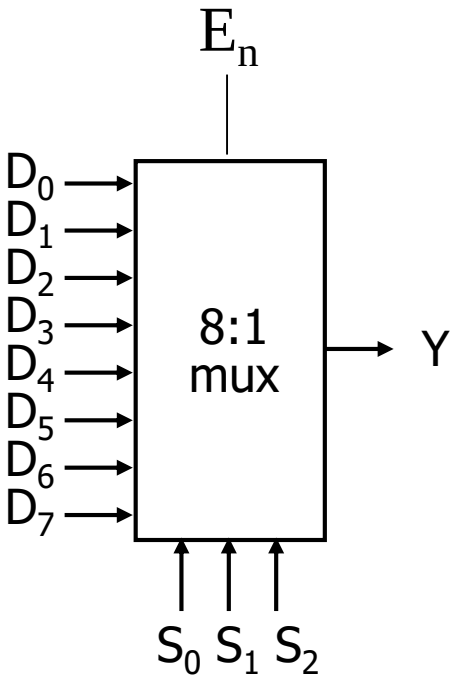
S_1	S_0	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

Functional truth table



$$Y = S_1'S_0'D_0 + S_1'S_0D_1 + S_1S_0'D_2 + S_1S_0D_3$$

8x1 Multiplexer



In the 8 to 1 multiplexer, there are total eight inputs, 3 selection lines, i.e., S0, Sland S2 and single output, i.e., Y. On the basis of the combination of inputs that are present at the selection lines S0, Sl, and S2, one of these 8 inputs are connected to the output. The block diagram and the truth table of the 8×1 multiplexer are given below.

Functional truth table

S_2	S_1	S_0	Y
0	0	0	D_0
0	0	1	D_1
0	1	0	D_2
0	1	1	D_3
1	0	0	D_4
1	0	1	D_5
1	1	0	D_6
1	1	1	D_7

The Boolean expression for an 8-to-1-line multiplexer is:

$$f= (D_0\overline{S}_2\overline{S}_1\overline{S}_0 + D_1\overline{S}_2\overline{S}_1S_0 + D_2\overline{S}_2S_1\overline{S}_0 + D_3\overline{S}_2S_1S_0 + D_4S_2\overline{S}_1\overline{S}_0 + D_5S_2\overline{S}_1S_0 + D_6S_2S_1\overline{S}_0 + D_7S_2S_1S_0).$$

MUX Tree

Mux tree is used to obtain higher order MUX using Lower order MUX

to make a p:1 multiplexer using only a q:1 multiplexer. The general methodology to tackle such problems is to divide n by m until we get 1 and find the number of stages and the number of multiplexers required.

$p/q = k1$ Stage 1

$k1/q = k2$ Stage 2

$k2/q = k3$ Stage 3 : :

$k_{n-1}/q = k_n = 1$ Stage n

total number of multiplexers $= k_1+k_2+\ldots+k_{n-1}+1$

MUX Tree

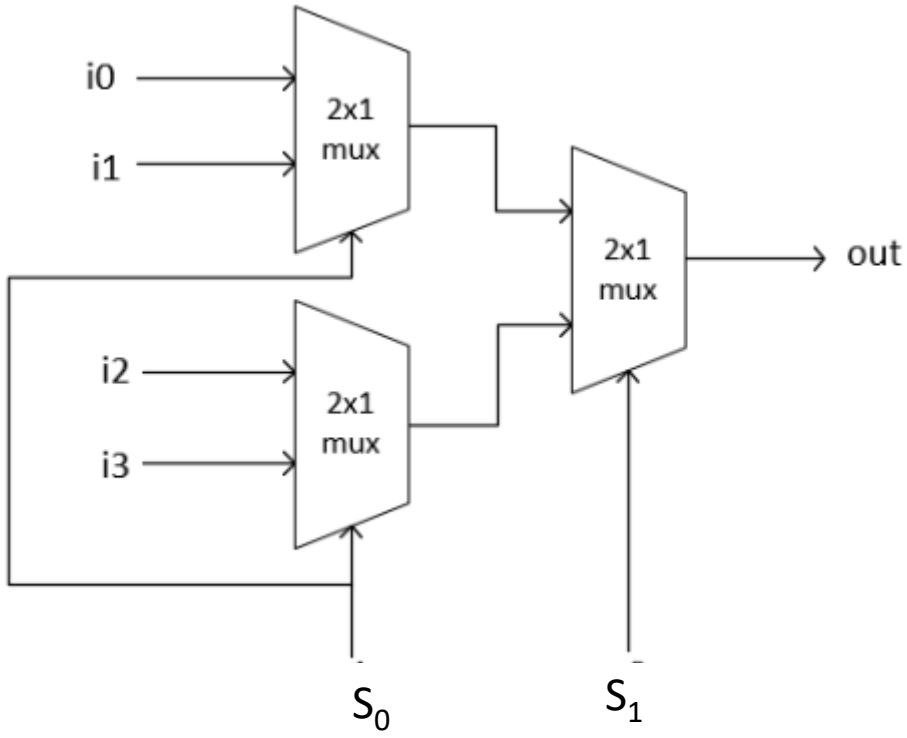
How many 4x1 MUX required to get 8x1 MUX?

How many 2x1 MUX required to get 32x1 MUX?

How many 4x1 MUX required to get 16x1 MUX?

How many 2x1 MUX required to get 16x1 MUX?

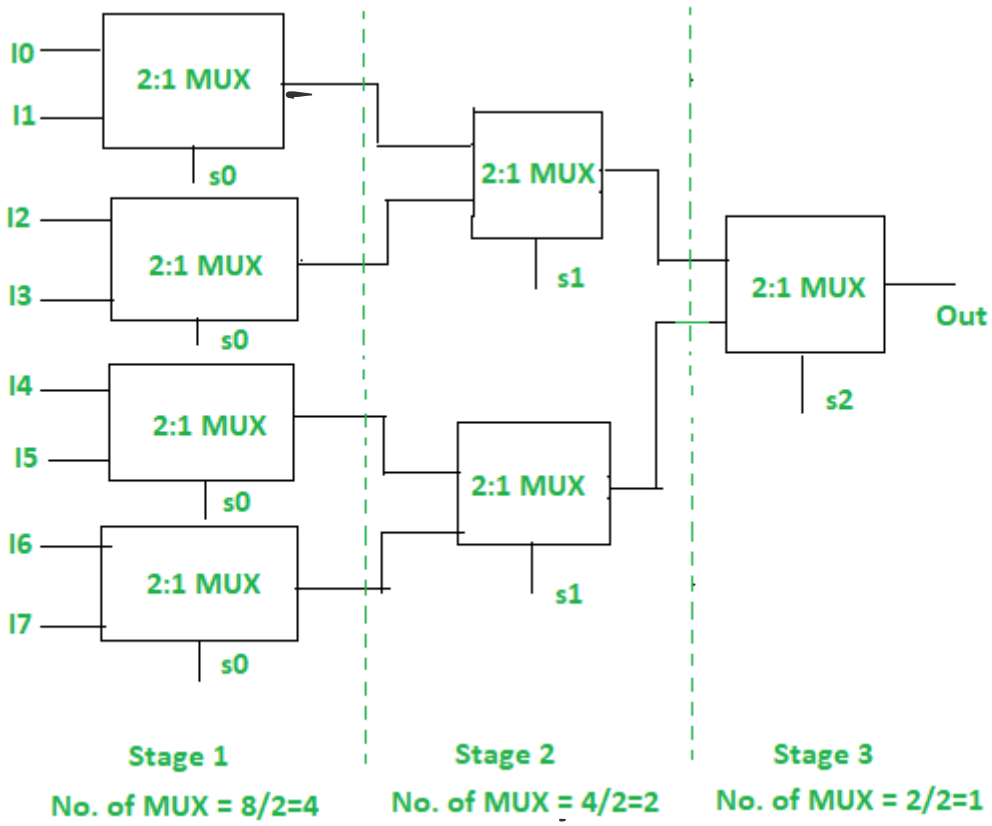
4x1 MUX using 2x1 MUX



Functional truth table

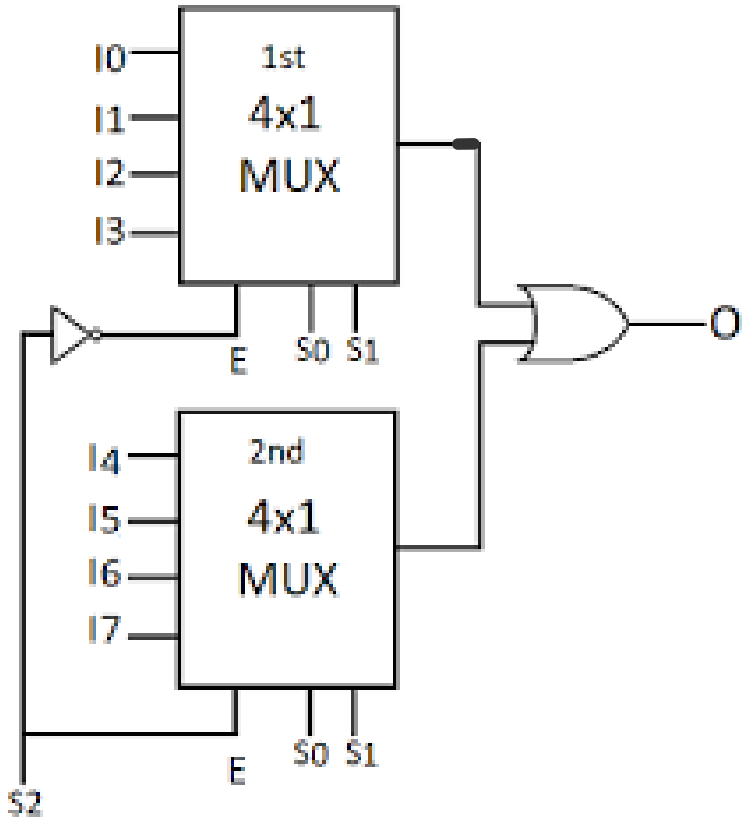
S_1	S_0	Y	
0	0	I_0	
0	1	I_1	
1	0	I_2	
1	1	I_3	

8x1 MUX using 2x1 MUX



S_2	S_1	S_0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

8x1 MUX using 4x1 MUX



Functional truth table

S_2	S_1	S_0	Y	
0	0	0	I_0	1st Mux is enabled
0	0	1	I_1	
0	1	0	I_2	
0	1	1	I_3	
1	0	0	I_4	2nd Mux is enabled
1	0	1	I_5	
1	1	0	I_6	
1	1	1	I_7	

MUX Example

- Example-1: Implement the following function with a multiplexer: $F(A,B,C) = \sum(1,3,5,6)$

  - If a Boolean function consists $n+1$ variables,
    - Take n of these variables and connect them to control/selection lines.
    - Remaining single variable is used for inputs.
      - Either MSB or LSB is choose for inputs
      - Choosing MSB is easier than LSB

Implement Boolean function $F(A,B,C) = \sum(1,3,5,6)$

- Choosing, B and C for selection lines and A (e.g., MSB) as inputs.
  - 1st row listed minterms where A complemented.
  - 2nd row listed minterms where A uncomplemented.
  - If two minterms in a column
    - Not circled, apply 0 as MUX input.
    - Circled, apply 1 as MUX input.
  - If bottom minterm circled and top not, apply A as MUX input.
  - For opposite case, apply A'

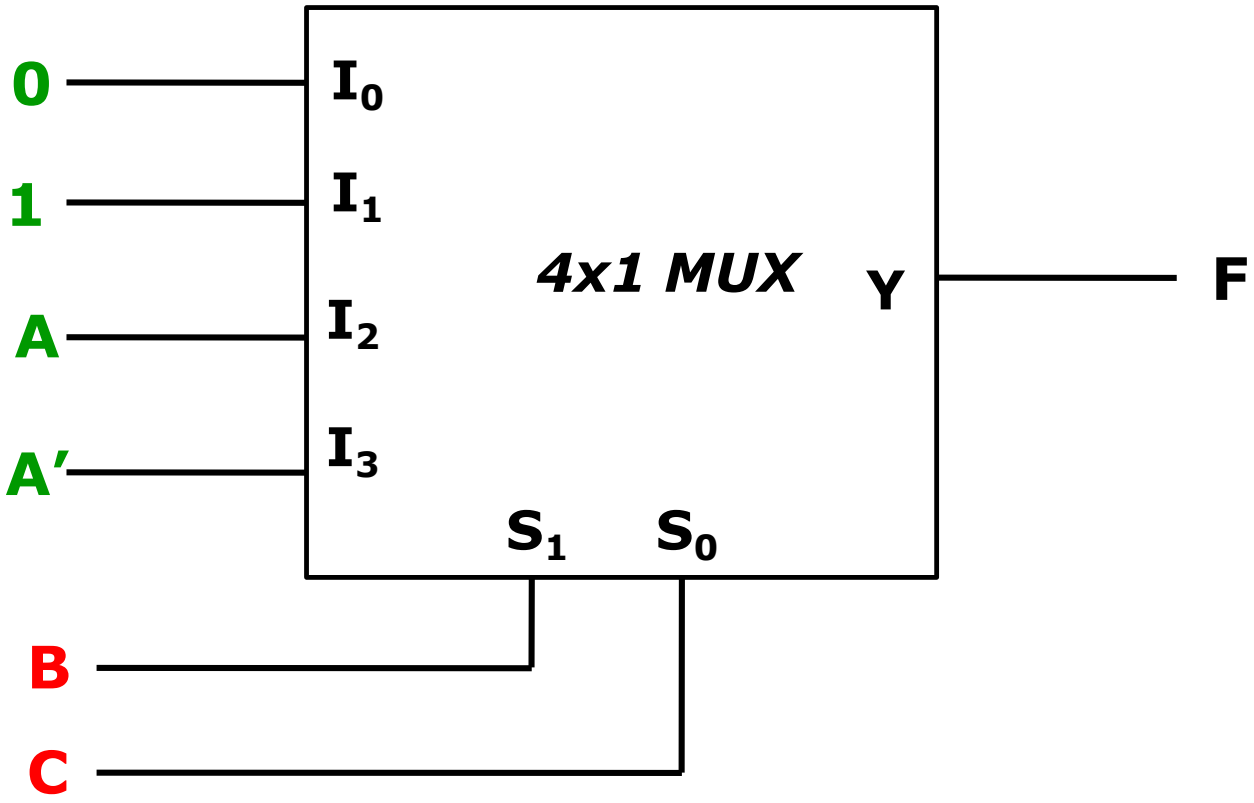
Implementation Table

	I_0	I_1	I_2	I_3
A'	0	(1)	2	(3)
A	4	(5)	(6)	7
	0	1	A	A'

Truth Table

minterm	A	B	C	F
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0

Implement Boolean function $F(A,B,C) = \sum(1,3,5,6)$



Implementation Table

	I_0	I_1	I_2	I_3
A'	0	1	2	3
A	4	5	6	7
	0	1	A	A'

Multiplexer implementation

Implement Boolean function $F(A,B,C) = \sum(1,3,5,6)$

- Choosing, LSB as inputs

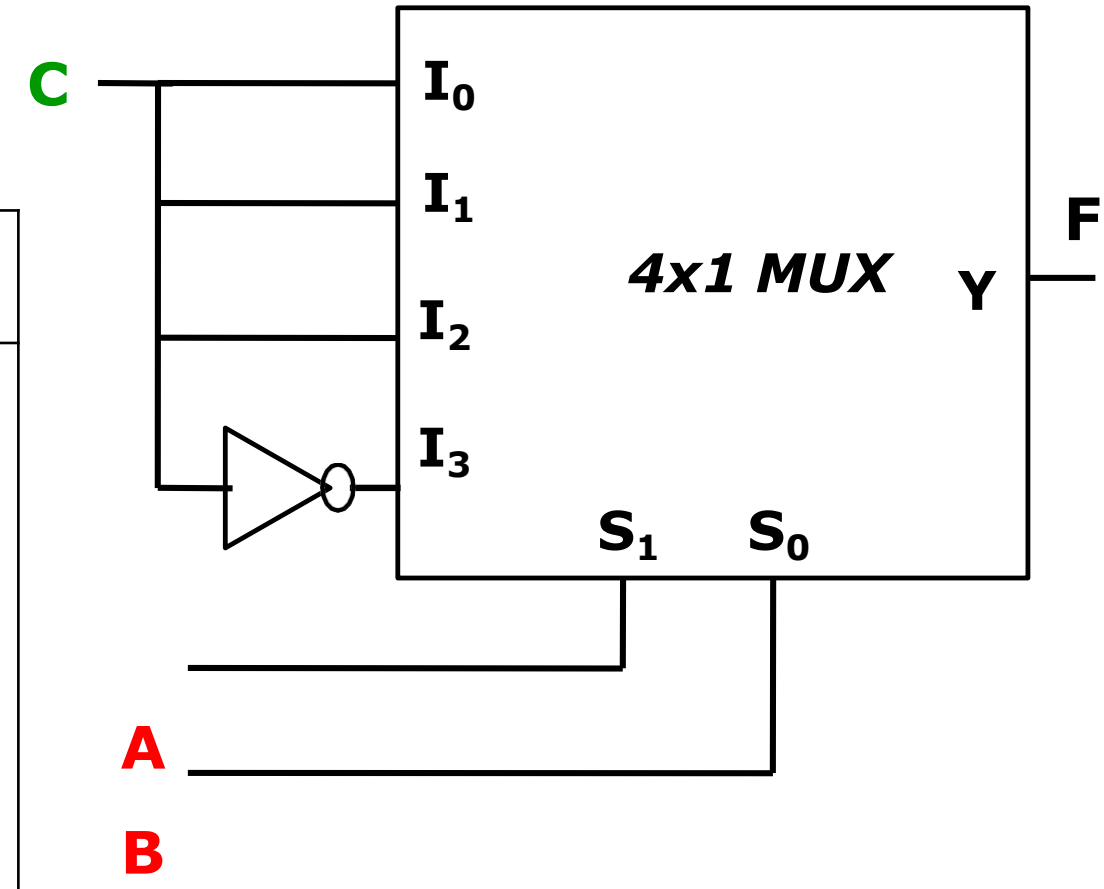
$F(A,B,C) = \Sigma(1,3,5,6)$

Implementation Table

	I_0	I_1	I_2	I_3
C′	0	2	4	(6)
C	(1)	(3)	(5)	7
	C	C	C	C′

Truth Table

minterm	A	B	C	F
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0



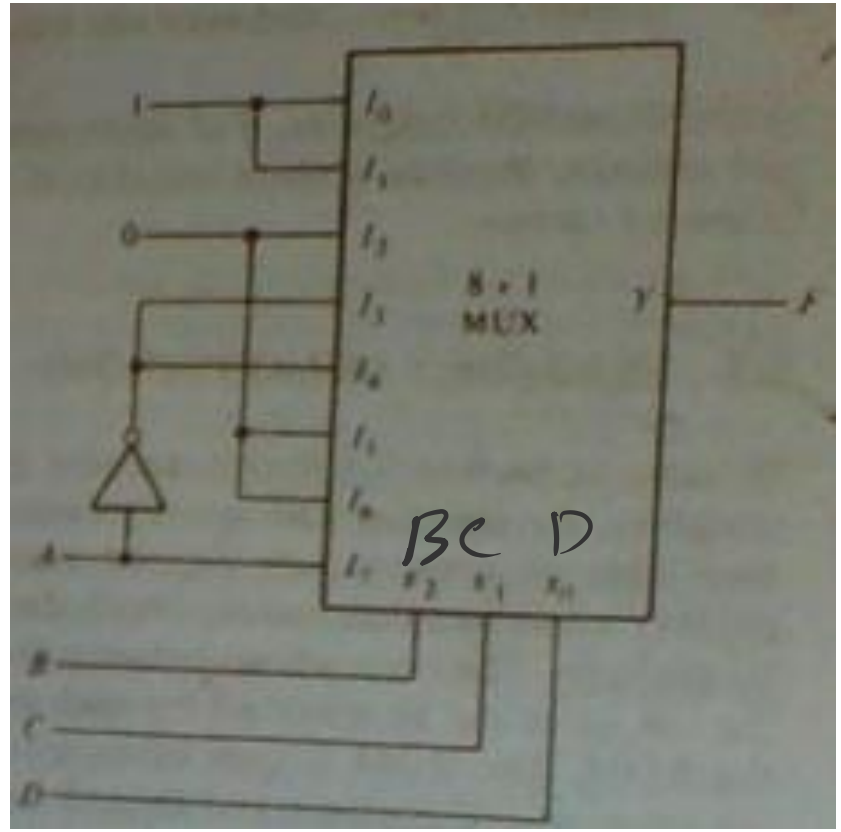
Multiplexer implementation

Implement Boolean function with a multiplexer

$F(A,B,C,D) = \sum(0,1,3,4,8,9,15)$

- Choosing, MSB or A as inputs

	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7
A′	0	1	2	3	4	5	6	7
A	8	9	10	11	12	13	14	15
	1	**1**	**0**	**A′**	**A′**	**0**	**0**	**A**



Multiplexer implementation

Implement Full adder with a 4x1 multiplexer

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$S(x,y,z)=\Sigma(1,2,4,7)$

$C(x,y,z)=\Sigma(3,5,6,7)$

For S

	I_0	I_1	I_2	I_3
x′	0	1	2	3
x	4	5	6	7
	x	**x′**	**x′**	**x**

For C

	I_0	I_1	I_2	I_3
x′	0	1	2	3
x	4	5	6	7
	0	**x**	**x**	**1**

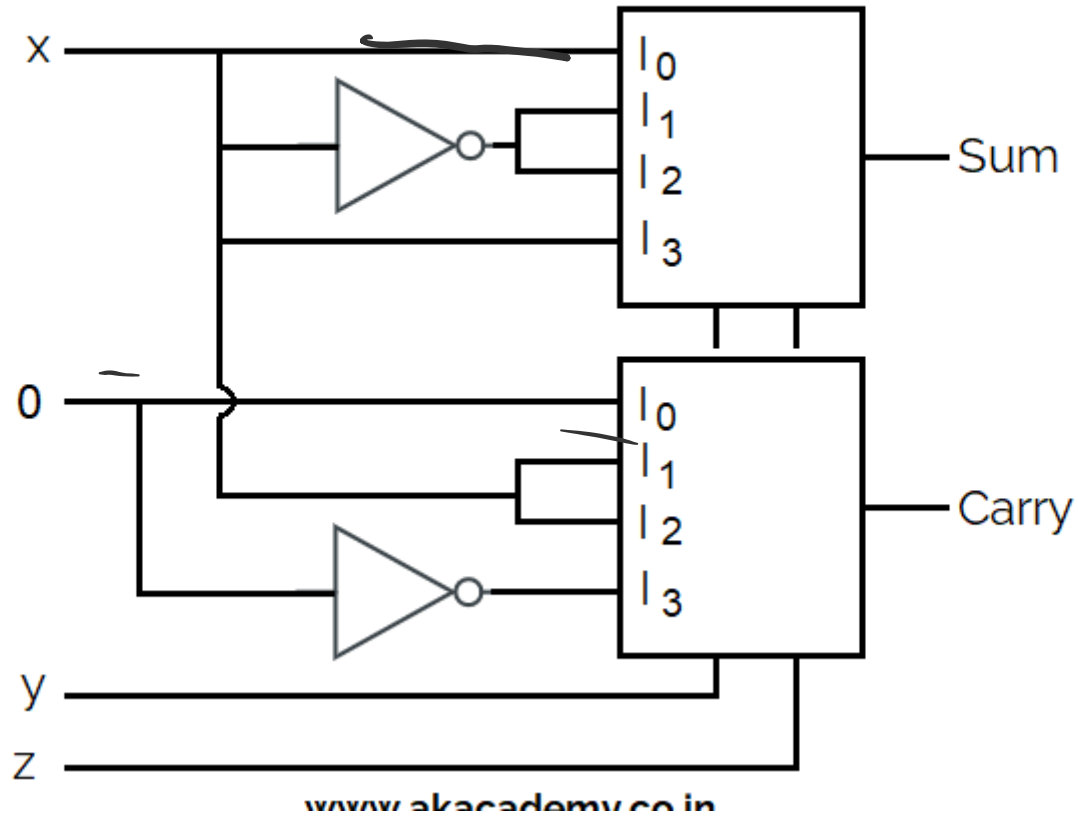
Implement Full adder with a 4x1 multiplexer

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$S(x,y,z)=\Sigma(1,2,4,7)$

$C(x,y,z)=\Sigma(3,5,6,7)$

Logic Diagram of FUll Adder Using 4X1 Multiplexers



Implement full adder with a 2x1 multiplexer

	x	y	z	C	S
0	0	0	0	0	0
1	0	0	1	0	1
2	0	1	0	0	1
3	0	1	1	1	0
4	1	0	0	0	1
5	1	0	1	1	0
6	1	1	0	1	0
7	1	1	1	1	1

$S(x,y,z)=\Sigma(1,2,4,7)$

$C(x,y,z)=\Sigma(3,5,6,7)$

For S

	I0	I1
X'Y'	0	1
X'Y	2	3
XY'	4	5
XY	6	7

For S

$I_0=X'Y+XY'$

$I_1=X'Y'+XY=(X'Y+XY')'$

For C

	I0	I1
X'Y'	0	1
X'Y	2	3
XY'	4	5
XY	6	7

For C

$I_0=XY$

$I_1=X'Y+XY'+XY$

$=X'Y+X(Y'+Y)$

$=X'Y+X$

$=(X'+X)(X+Y)$

$=X+Y$

Implement full adder with a 2x1 multiplexer

	x	y	z	C	S
0	0	0	0	0	0
1	0	0	1	0	1
2	0	1	0	0	1
3	0	1	1	1	0
4	1	0	0	0	1
5	1	0	1	1	0
6	1	1	0	1	0
7	1	1	1	1	1

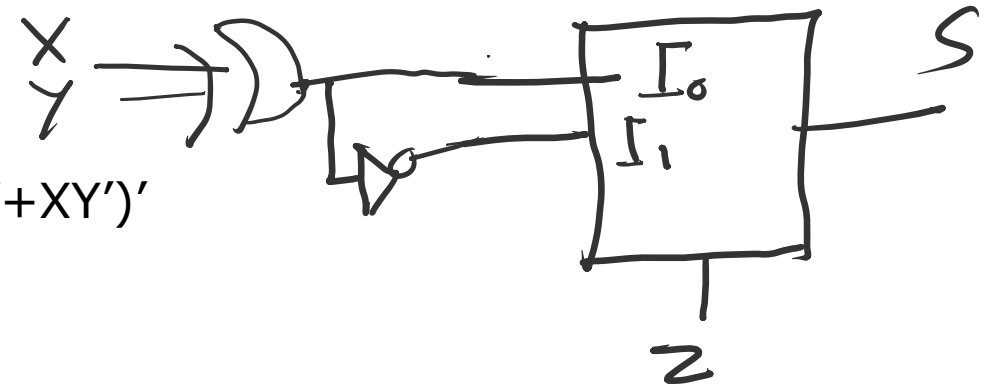
$S(x,y,z)=\Sigma(1,2,4,7)$

$C(x,y,z)=\Sigma(3,5,6,7)$

For S

$I_0=X'Y+XY'$

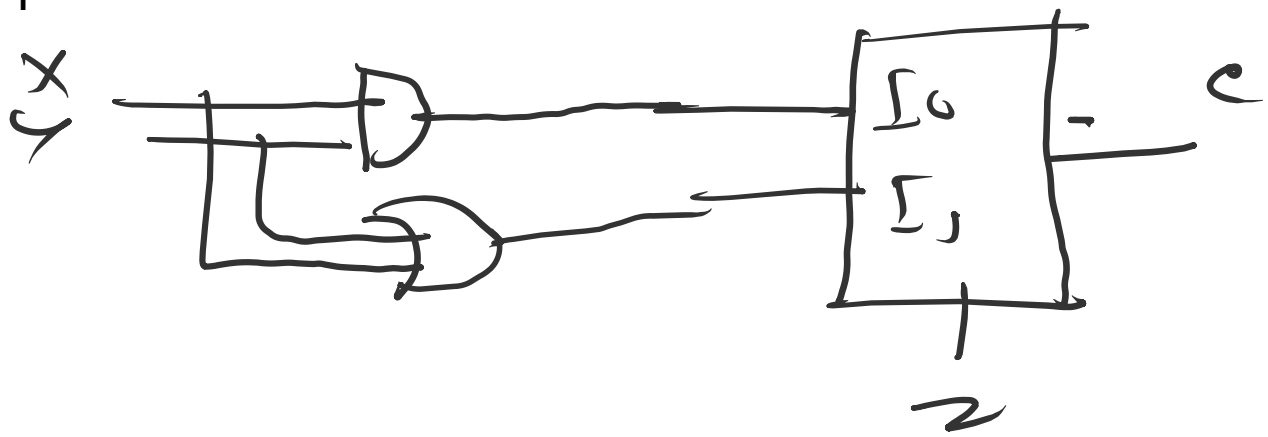
$I_1=X'Y'+XY=(X'Y+XY')'$



For C

$I_0=XY$

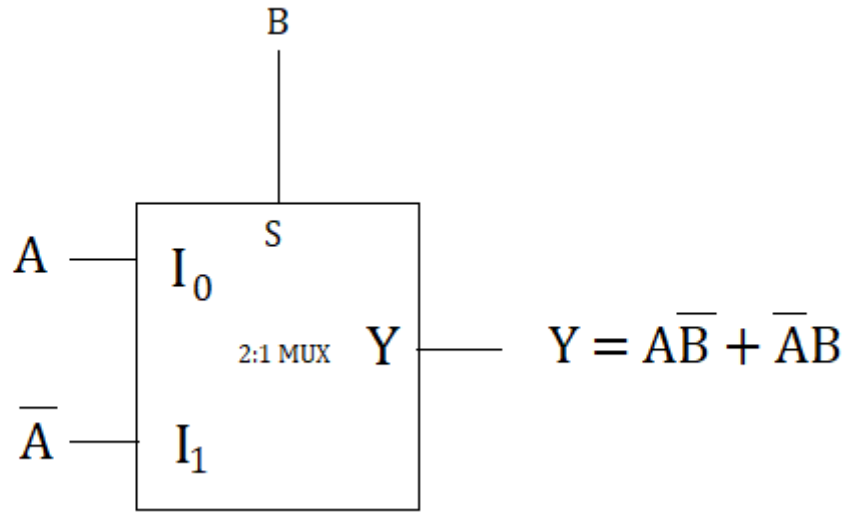
$I_1=X+Y$



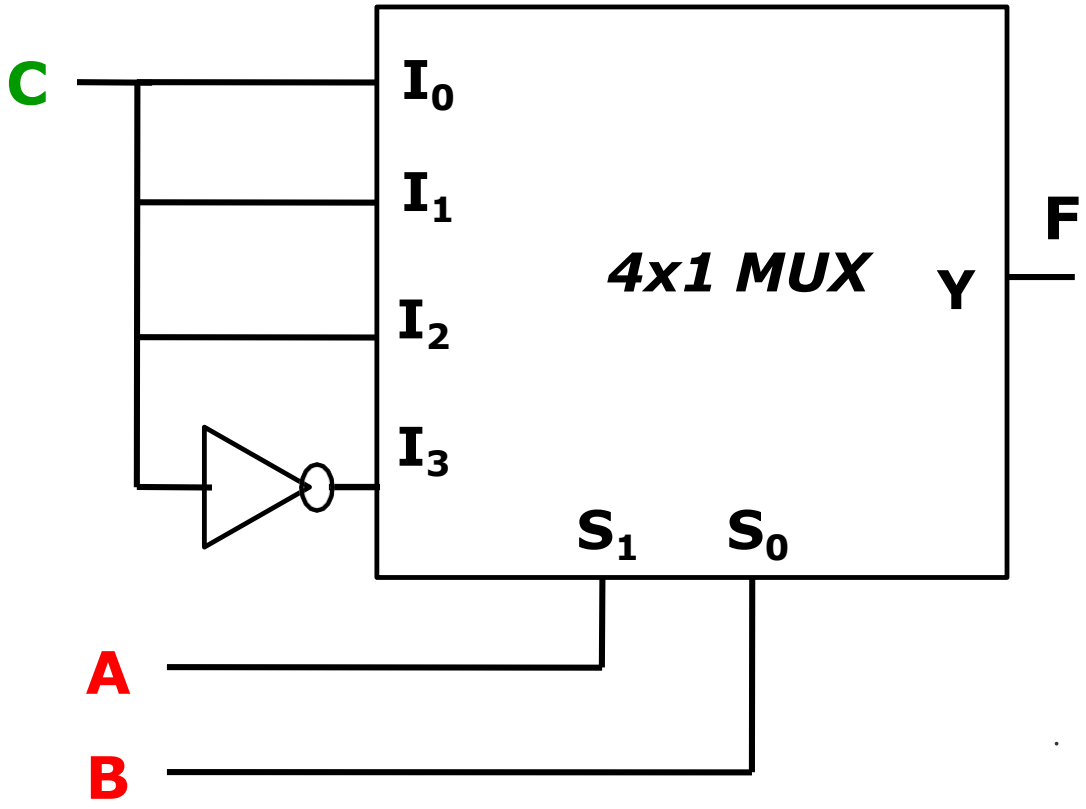
Implement X-OR gate with a 2x1 multiplexer

	A	B	Y
0	0	0	0
1	0	1	1
2	1	0	1
3	1	1	0

	I_0	I_1
A'	0	1
A	2	3
	A	A'



Derive the expression of the function



Multiplexer implementation

$Y = S_0'S_1'I_0 + S_0'S_1I_1 + S_0S_1'I_2 + S_0SI_3$

Y=A'B'C+A'BC+AB'C+ABC'

Y=A'C(B'+B)+AB'C+ABC')

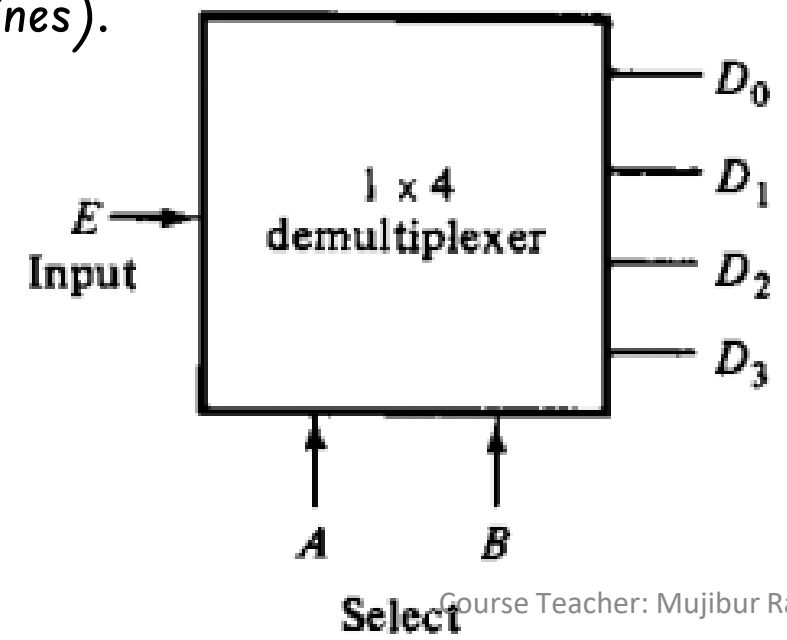
Y=C(A'+AB')+ABC'

Y=C(A'+B')+ABC'

=CA'+CB'+ABC'

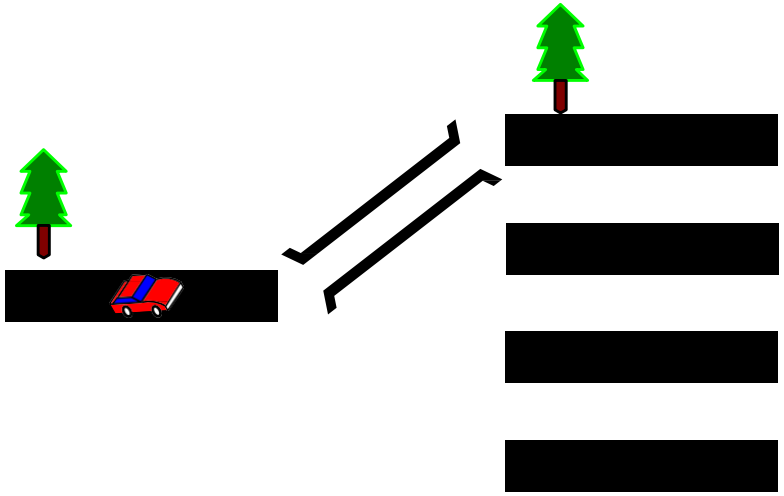
DeMultiplexers

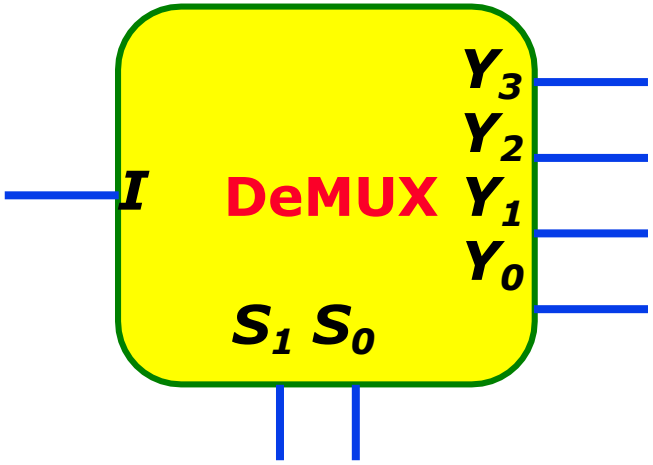
- A circuit that receives information on a single line and transmit this information on one of 2^n possible output lines.
  - Selection of a specific output line is controlled by bit values of n selection lines.
  - To be able to select among n inputs, $\log_2 n$ control lines are needed (2^n input lines and n selection lines).



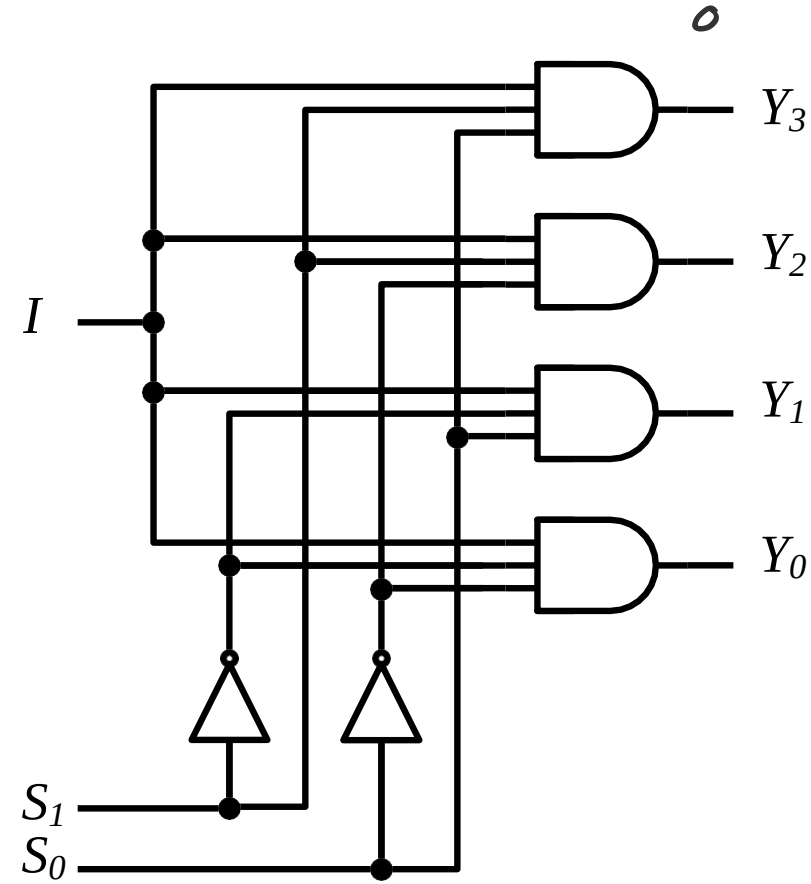
Block diagram of a Demultiplexer

1x4 DeMultiplexer





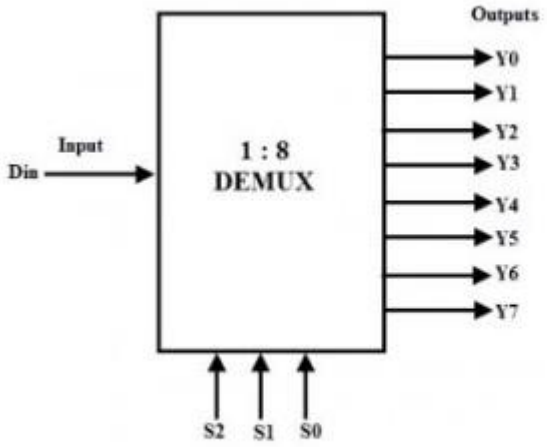
S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	I
0	1	0	0	I	0
1	0	0	I	0	0
1	1	I	0	0	0



$Y_3 = I.S_1S_0$ $Y_2 = I.S_1S_0'$

$Y_1 = I.S_1'S_0$ $Y_0 = I.S_1'S_0'$

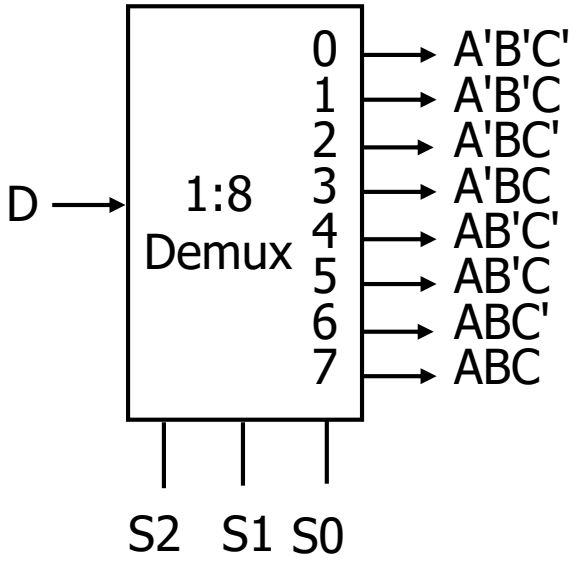
1x8 DeMultiplexer



1 to 8 Demux Block Diagram

Data Input	Select Inputs			Outputs							
D	S_2	S_1	S_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
D	0	0	0	0	0	0	0	0	0	0	D
D	0	0	1	0	0	0	0	0	0	D	0
D	0	1	0	0	0	0	0	0	D	0	0
D	0	1	1	0	0	0	0	D	0	0	0
D	1	0	0	0	0	0	D	0	0	0	0
D	1	0	1	0	0	D	0	0	0	0	0
D	1	1	0	0	D	0	0	0	0	0	0
D	1	1	1	D	0	0	0	0	0	0	0

1 to 8 Demux Truth Table

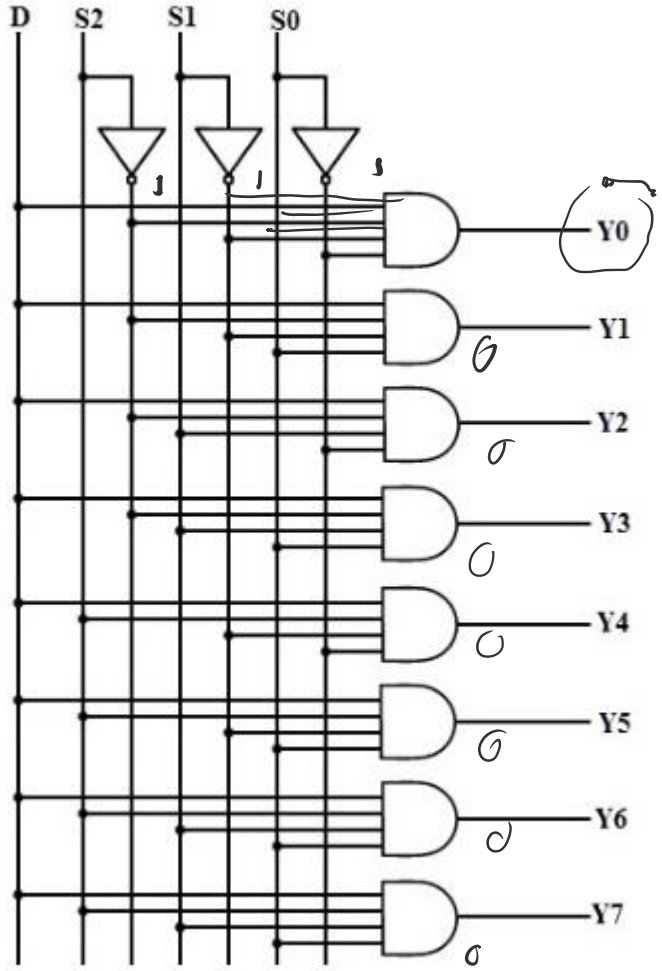


2-to-4 Decoder can be used as 1-to-4 DMUX

S2=A

S1=B

S0=C



Demultiplexers as general-purpose logic

Example

$F1 = A'BC'D + A'B'CD + ABCD$

$F1 = \sum(3,5,15)$

$F2 = ABC'D' + ABC$

$F_2 = \sum(12,14,15)$

$F3 = (A'+B'+C'+D')$

$F3 = (ABCD)'$